1. JUSTIFICACIÓN

Durante la práctica 5 hemos descrito el protocolo de comunicaciones ModBus, y hemos implementado sendas aplicaciones, en entorno arduino para recibir una trama ModBus y otra en entorno Java que la envía. Hemos usado el comando 05_H (**Escritura de una señal discreta de salida**), la Figura 1 muestra la trama enviada desde el maestro y la Figura 2 la enviada desde el dispositivo esclavo. Para poder llevar a cabo la operación, introdujimos la clase "ModbusSlave".

Pregunta : Maestro → Esclavo							
Dirección (1 byte)	Función (1 byte)	Datos				Ctrl. Err. (CRC)	
		Dirección Comienzo		Número de registros			
		Byte Alto	Byte Bajo	Byte Alto	Byte Bajo		
01_H	05_H	00_H	00_H	FF_H	00_H	$8C_H$	$3A_H$

Figura 1.- Trama Maestro-Esclavo enviada en la práctica 5

Pregunta : Esclavo → Maestro							
Dirección (1 byte)	Función (1 byte)	Datos				Ctrl. Err. (CRC)	
		Dirección Comienzo		Número de registros			
		Byte Alto	Byte Bajo	Byte Alto	Byte Bajo		
01_H	05_H	00_H	00_H	FF_H	00_H	$8C_H$	$3A_H$

Figura 2.- Trama Esclavo-Maestra recibida en la práctica 5

Durante el desarrollo de la práctica 5, también estudiamos, que ModBus, aunque no tiene limitaciones en cuanto a los tipos de datos, se estructura para manejar cuatro tipos de datos fundamentales. Como se muestra en la Tabla 1, estos se distribuyen en cuatro bloques conceptuales.

Tabla 1.- Modelos de datos ModBus

Bloque de Memoria	Tipos de Datos	Acceso de Maestro	Acceso de Esclavo
Salidas Digitales	Booleano	Lectura/Escritura	Lectura/Escritura
Entradas Digitales	Booleano	Sólo Lectura	Lectura/Escritura
Registros de Retención	Palabra Sin Signo	Lectura/Escritura	Lectura/Escritura
Registros de Entrada	Palabra Sin Signo	Sólo Lectura	Lectura/Escritura

Para manejar las tramas descritas en las Figura 1 y Figura 2, en el proyecto "P5_Comunicaciones_ModBus_Sol_Arduino" se introdujo la definición del banco de registros donde se mantiene una copia del estado de las salidas digitales usadas en el esclavo,

```
uint16_t     Cregs[MB_O_COILS];        //Registros para "Dicrete Output Coils"
```

En el caso de "DomoBoard", hasta ahora, hemos identificado dos dispositivos digitales de salida, RELE y

TRIAC. Como muestra la Tabla 1, el Maestro tiene acceso de lectura/escritura sobre este banco de registros de salidas digitales. Este banco de registros, es leído para monitorizar el estado de una salida digital (coil) y se escribe sobre él para activar esa misma salida, es por lo que al comando 05_H lo estamos llamando **WRITECOIL**. En la solución propuesta, una vez escrito el registro, se genera un evento "**writecoil**" con objeto de que el nuevo valor, se vea reflejado en la salida. Por tanto, para activar una salida, escribimos sobre el registro correspondiente de salida digital.

Para seguir avanzando en la gestión del dispositivo esclavo "DomoBoard", vamos a incluir en esta práctica, la gestión de un nuevo tipo de datos de los mostrado en la Tabla 1, **Entradas Digitales**. Para ello, debemos definir el correspondiente banco de registros de entrada, y ser cuidadosos, solo el esclavo puede escribir en él. Este nuevo banco de registro debe contener una copia de los dispositivos digitales de entrada que queremos monitorizar.

Hasta ahora, los pulsadores usados en "domoboard" son las únicas entradas digitales que hemos usado en las prácticas y, por tanto, son las que vamos a usar como "Entradas Digitales Modbus". En este sentido, usaremos las comunicaciones modbus para conocer el estado de los pulsadores en cada instante. Cada vez que pulsemos o soltemos uno de los pulsadores, el estado correspondiente del mismo, debe reflejarse en el correspondiente registro modbus, lo que permitirá al dispositivo maestro conocer el estado actual de los pulsadores con solo leer el banco de registro de "**Entradas Digitales"**.

Así pues, a modo de resumen, de los dispositivos introducidos en prácticas anteriores, podemos identificar como entradas digitales, *los pulsadores*. Y para que estos dispositivos de entrada puedan ser gestionados por ModBus, el banco de registros para **entradas digitales**, debe contener en todo momento, una copia del estado de los pulsadores: **Pulsado/No Pulsado**.

Si recordáis, durante la práctica 5 usamos el comando modbus (WRITE_COIL = 5) para actuar sobre el dispositivo esclavo, enviando comandos para activar/desactivar salidas digitales. En esta práctica pretendemos ir un paso más allá, y recibir también información desde los dispositivos conectados, que se mostraran en la interface java, y así mantener actualizado el estado del sistema. Para ello, debéis implementar dos nuevas instrucciones modbus:

- READ_COILS = 1 -- para leer el estado (banco de registros digitales de salida) de los dispositivos digitales de salida
- READ_INPUT_DISCRETE = 2 – para leer el estado (banco de registros digitales de entrada) de los dispositivos digitales de entrada

Recordad, que como indica la Tabla 1, los dispositivos digitales de entrada son mantenidos en un banco de registros distinto a los dispositivos digitales de salida.

2. MONTAJE

Partiendo del proyecto eclipse java entregado como Anexo 1 de la práctica 5, y según lo definido anteriormente en este documento, el trabajo a realizar será:

I. Debe definir un nuevo banco de registros para gestionar los dispositivos digitales de entrada.

II. Modificar la aplicación arduino para que el nuevo banco de registros definido, refleje en todo momento el estado de los pulsadores (Btn.1, Btn.2 y btn_opt).

III. Implementar en la aplicación Java un sistema que muestre el estado del esclavo. Estado de los pulsadores, el estado del relé y el estado del triac.